

Moduł **reservations** dla Open Mercato: propozycja kontrybucji

CZĘŚĆ I: WARSTWA BIZNESOWA

1. Po co ten moduł

Budujemy do własnej aplikacji silnik rezerwacji: rezerwacja konkretnego zasobu w oknie czasowym, wykrywanie konfliktów, harmonogram. Open Mercato dziś tego nie ma, a potrzebuje tego każdy klient, który rezerwuje cokolwiek na czas, niezależnie od branży.

Ponieważ ta logika nie jest specyficzna dla naszej aplikacji ani żadnej pojedynczej branży, jest naturalnym kandydatem na reużywalny moduł rdzeniowy Open Mercato (**reservations**). Proponujemy zbudować go od razu generycznie, tak by konkretne produkty tylko go konfigurowały i rozszerzały. Open Mercato dostaje brakujące ogniwo i projekt referencyjny; kolejni klienci z potrzebą rezerwacji zasobów dziedziczą gotowy silnik.

2. Co moduł dostarcza

Moduł odpowiada na jedno pytanie: **"zarezerwuj ten konkretny zasób na ten cel w tym oknie czasowym i ostrzeż, jeśli coś się nakłada"**. Konkretnie dostarcza:

- **Rezerwację zasobu na cel w czasie.** Brakujące ogniwo: powiązanie konkretnego zasobu z konkretnym celem w konkretnym oknie, z własną semantyką (status, czas trwania, deadline startu, stan nieumieszczony).
- **Jedno źródło prawdy o zajętości.** Rezerwacja spływa do plannera jako okno "zajęte", więc dowolny moduł może zapytać "czy podmiot jest zajęty" bez znajomości domeny rezerwacji.
- **Wykrywanie konfliktów.** Nakładanie rezerwacji tego samego podmiotu oraz wpadnięcie w okno niedostępności. Sygnalizowane wizualnie i zdarzeniem do alertów.
- **Wczesne ostrzeganie o braku pokrycia.** "Za X dni cel zostaje bez zasobu"; próg konfigurowalny.
- **Widoczność wolnych podmiotów.** Harmonogram pokazuje dostępność pozostałych podmiotów tego samego typu w danym oknie, więc widać, co jest wolne, bez osobnej funkcji rekomendującej zamiennik.
- **Oś czasu (widok resource-lane).** Wiersz per podmiot, rezerwacje jako paski, konflikt oznaczony wizualnie.

- **Kalendarz roboczy organizacji.** Liczenie czasu trwania w dniach roboczych z uwzględnieniem dni wolnych i świąt.

3. Jak można go wykorzystać

Ten sam mechanizm obsługuje między innymi:

- **sprzęt** (koparka, wózek widłowy, aparat USG, sprzęt eventowy),
- **przestrzeń** (sala konferencyjna, stanowisko, boks serwisowy),
- **wizyty i usługi** (termin u klienta, slot serwisowy),
- **zmiany i grafiki** (obsada na zmianę).

Konkretny produkt dokłada tylko swoją domenę (co to za cel, jakie pola, jakie alerty), a rdzeń dostarcza rezerwację, konflikty, oś czasu i zajętość. Dzięki temu koszt zbudowania kolejnego produktu rezerwacyjnego na Open Mercato spada do warstwy produktowej.

4. Zakres MVP

Poniżej co faktycznie wchodzi w MVP modułu, a co okrajamy.

4.1 Wchodzi w całości

- **Dostępność, harmonogram, konflikty:** jądro modułu. Encja rezerwacji, wykrywanie konfliktów, wczesne ostrzeganie o braku pokrycia.
- **Kalendarz roboczy per organizacja:** konfigurowalne dni wolne tygodnia plus konkretne święta. Silnik liczy czas trwania w dniach roboczych, miękko (nie blokuje pracy w dzień wolny, tylko urealnia liczenie). Globalny dla organizacji, bez wyjątków per podmiot w MVP.
- **Cel rezerwacji:** lekki, generyczny; produkt rozszerza go polami własnymi.
- **Oś czasu (widok resource-lane):** wiersz per podmiot. Skala dzienna w MVP.

4.2 Konfigurowalne parametry w MVP

Niepełny silnik reguł, tylko parametry modułu:

- próg wyprzedzenia ostrzeżenia o braku pokrycia (ile dni przed brakiem rezerwacji ostrzegać),
- kalendarz roboczy organizacji (dni wolne tygodnia plus święta),
- granularność czasu ustawiona na dni; przejście na mniejszą granularność (godziny) nastąpi w późniejszych iteracjach. Model danych jest na to gotowy od początku, więc nie wymaga zmian w danych przy dokładaniu skali godzinowej.

4.3 Okrajamy (post-MVP)

- **Silnik reguł:** pełna konfigurowalność warunków i akcji per klient bez kodu. W MVP konfigurowalne są tylko parametry z 4.2.
- **Dopasowanie kwalifikacji** (luka uprawnień, dopasowanie podmiotu do wymagań celu).
- **Zgłoszenia i przydział dwukierunkowy** (kierunek "od celu", obieg akceptacji). MVP działa "od zasobu".
- **Kaskadowe przełożenie** (push/pull, przesuwanie łańcucha rezerwacji).
- **Przekroczenie pojemności** (rezerwacja sali/puli/miejsc, gdzie konflikt to suma rezerwacji ponad pojemność). W MVP konflikt to proste nakładanie; model nie zamyka drogi do pojemności.
- **Godzinowa skala osi czasu.** Model gotowy, UI i liczenie godzin pracy dokładamy później.
- **Wyjątki w kalendarzu roboczym per podmiot lub cel.** W MVP kalendarz jest wspólny dla organizacji.

Uwaga na odróżnienie od kaskadowego przełożenia: widoczność wolnych podmiotów **zostaje w MVP**. Przy konflikcie albo luce na dany termin harmonogram i tak pokazuje dostępność pozostałych podmiotów tego samego typu, więc dyspozytor widzi, co jest wolne, i decyduje sam. To wynika z samego widoku dostępności, nie jest osobną funkcją rekomendującą zamiennik, i nie należy mylić tego z kaskadą (przesunięciem całego łańcucha rezerwacji), która jest post-MVP.

CZĘŚĆ II: WARSTWA TECHNICZNA

5. Na czym moduł stoi (istniejące filary Open Mercato)

Moduł nie buduje fundamentu od zera. Opiera się na trzech istniejących modułach Open Mercato i tylko dokłada brakującą warstwę rezerwacji.

- **resources** dostarcza rejestr zasobów i egzemplarzy. To jest "co rezerwujemy". Twarda zależność, nie budujemy własnego rejestru.
- **planner** dostarcza okna dostępności i niedostępności oraz **plannerAvailabilityService** liczący dostępność w czasie. To jest "kiedy podmiot jest w ogóle dostępny".
- **events + notifications** dostarczają mechanizm alertów. Moduł tylko emituje typowane zdarzenia (np. wykryto konflikt), a dostarczanie powiadomień obsługuje istniejąca infrastruktura Open Mercato.

Do tego fundament platformy (RBAC, słowniki, wyszukiwanie, audyt) jest gotowy w Open Mercato i wliczony jako baza.

6. Kluczowa decyzja architektoniczna: wpięcie w planner

To był główny znak zapytania na starcie: `planner` ma zamknięty enum `subjectType` (`member` | `resource` | `ruleset`), więc pytanie brzmiało, czy generyczny podmiot rezerwacji da się podłączyć bez zmian w rdzeniu planner'a.

Odpowiedź: da się, bez ruszania enuma. Mechanizm wpięcia:

- **Sprzęt** mapujemy wprost na `subjectType='resource'` (egzemplarz to encja modułu `resources`).
- **Wszystko inne**, co nie jest `member` ani `resource`, wpinamy przez `subjectType='ruleset'`: podmiot dostaje własny `PlannerAvailabilityRuleSet`, a reguły celują w jego id. Ruleset działa jak uniwersalny adapter, więc dowolny rejestr da się podpiąć bez zmian w plannerze.
- **Kalendarz roboczy organizacji** realizujemy tym samym wzorcem: dedykowany ruleset, którego id trzymamy w `configs`.

Konsekwencja dla modułu: rozszerzenie enuma planner'a **nie jest twardą zależnością**.

Wzorec działa na niezmienionym plannerze, co upraszcza kontrybucję i zdejmuje ryzyko procesowe (nie czekamy na zgodę na zmianę w rdzeniu planner'a).

7. Model danych rdzenia

7.1 Encja `assignment` (rezerwacja)

Rezerwacja to własna encja rdzenia, bo niesie semantykę, której okno planner'a nie pomieści.

Glosariusz: w tym module `assignment` oznacza rezerwację podmiotu (zasobu) na cel w oknie czasowym; nie chodzi o przypisanie uprawnień ani zadań.

Kluczowe pola: podmiot rezerwacji (`subjectType` + `subjectId`, spójne z plannerem), cel (`targetId`), okno czasowe (`startAt` / `endAt`, pełne znaczniki czasu), status (maszyna stanów), czas trwania w dniach roboczych, deadline startu, oraz referencja do reguły planner'a reprezentującej okno zajętości (`plannerRuleId`).

Stan nieumieszczony. Rdzeń zna rezerwację bez okna czasowego (bez `startAt` / `endAt`): rezerwacja istnieje jako zobowiązanie, ale nie leży jeszcze na osi. To świadoma część kontraktu rdzenia, nie detal produktu.

Zajętość: jedno źródło prawdy. Po umieszczeniu rezerwacji na osi zapisujemy odpowiadające okno "zajęte" w plannerze (przez ruleset). Dzięki temu planner pozostaje jedynym miejscem, które odpowiada na pytanie "czy podmiot jest zajęty" (serwisy, rezerwacje i kalendarz jedną drogą), a inne moduły widzą zajętość bez znajomości domeny rezerwacji.

7.2 Encja `assignment_target` (cel rezerwacji)

Lekki, generyczny cel rezerwacji. Aplikacja rozszerza go polami własnymi (np. priorytet, klient), rdzeń nie zna pojęcia "budowa".

Pola rdzenia: nazwa, opcjonalne okno czasowe celu, oraz **adres tekstowy**. Generycznej encji adresu w Open Mercato nie ma (istnieją trzy osobne kopie z twardymi relacjami ORM), więc cel trzyma własne pole adresu tekstowego wzorcem FK-id + snapshot, bez relacji `@ManyToOne` do `CustomerAddress`. Współrzędne i mapa są poza MVP.

7.3 Kontrakt rejestru (co musi mieć "rzecz", żeby dała się rezerwować)

Aby dowolny rejestr wszedł na harmonogram, jego encja musi spełniać:

- stabilne `id`,
- scope multi-tenant (`tenantId` / `organizationId` na podmiocie i regułach),
- wpięcie w planner (`subjectType` / `subjectId`, przez `resource` lub własny `ruleset`) niezbędne do pobrania okien niedostępności podmiotu; samo wpięcie nie jest używane przez silnik bezpośrednio, lecz stanowi źródło danych dla adaptera,
- kategorię do grupowania wierszy jako `DictionaryEntry` (z ikoną i kolorem),
- flagę aktywności / wycofania.

Silnik nie zna konkretnych typów (sprzęt, sala, wizyta, obsada). Konsument **mapuje swoje encje na generyczne wejście** silnika (`subjectType` + `subjectId` + okna czasowe), zamiast podawać konkretny obiekt swojej domeny. To jest warunek reużywalności: rdzeń widzi tylko generyczny podmiot i okna, produkt dokłada znaczenie.

8. Wykrywanie konfliktów

Silnik konfliktów i liczenie kalendarza roboczego to czyste funkcje. Przyjmują na wejściu zwykłe wartości: okna czasowe rezerwacji, okna dostępności/niedostępności, dni wolne kalendarza, i zwracają wynik jako dane (lista konfliktów, liczba dni roboczych). Bez I/O, bez zapisu, bez zależności od frameworka ani bazy. Dzięki temu ten sam kod działa po stronie serwera i w przeglądarce (podgląd konfliktu na żywo przy przeciąganiu paska na osi), a rdzeń jest w pełni przenaszalny, przenosi się między projektami bez zmian. Warstwy zależne od frameworka (zapis, odczyt, planner) otaczają ten rdzeń, nigdy go nie zanieczyszczają. Silnik nie odpytuje plannera bezpośrednio.

- **Definicja konfliktu w MVP:** nakładanie dwóch rezerwacji tego samego podmiotu w zachodzących terminach, albo wpadnięcie rezerwacji w okno niedostępności.
- **Mechanizm.** Planner nie jest wywoływany wewnątrz silnika. Adapter modułu odpytuje `plannerAvailabilityService.getMergedAvailabilityWindows` o okna dostępności podmiotu (reguły podawane na wejściu - serwis jest czystym wrapperem obliczeniowym) i przekazuje te okna na wejściu czystej funkcji wykrywania konfliktów, razem z rezerwacjami tego samego podmiotu. Granica jest wyraźna: **planner liczy „kiedy podmiot jest w ogóle dostępny”, silnik rezerwacji liczy „czy rezerwacje się nakładają”**, spina je cienka warstwa adaptera, nie sam rdzeń.
- **Wynik:** Konflikt jest zawsze sygnalizowany (wizualnie na osi + zdarzenie do alertów), nigdy nie blokuje zapisu- system podpowiada, człowiek decyduje.

Ułatwienie do rozważenia w OM. Skoro połączenie „czysty silnik + planner” powtórzy się u każdego klienta rezerwacyjnego, docelowo warto dostarczyć w OM gotowy port/adapter (np. `plannerAvailabilityAdapter`), który pobiera okna z planner'a i zwraca je w kształcie oczekiwanym przez silnik konfliktów. Klient dostaje wtedy wykrywanie konfliktów „po podłączeniu”, bez pisania integracji z plannerem od zera, a sam silnik pozostaje czysty i testowalny w oderwaniu od planner'a.

Uwaga poprawnościowa. Wykrywanie konfliktów po zapisie musi czytać **zatwierdzony (committed) stan bazy**, a nie indeks wyszukiwania query engine. Indeks konverguje asynchronicznie i może jeszcze nie zawierać dopiero co zapisanej rezerwacji, która wywołała sprawdzenie. Dlatego detekcja jest **osobnym odczytem reaktywnym** (po zdarzeniu zapisu place/move/resize), nie częścią samej komendy zapisu. Odczyt uruchamia czystą funkcję i zwraca listę konfliktów; sygnał (zdarzenie do alertów) idzie osobno. Pojemność jest świadomie pominięta w MVP (patrz 4.3).

9. Oś czasu

W `@open-mercato/ui/backend/schedule` istnieje kompletny `ScheduleView` (oparty na react-big-calendar) z widokami dzień/tydzień/miesiąc i modelem `ScheduleItem` (ma już `subjectType` / `subjectId` / `status` / `color`). Czego brakuje: widoku **resource-lane** (wiersz per podmiot), a to jest sedno osi czasu rezerwacji.

Plan: reużycie istniejącego `ScheduleView` (react-big-calendar) z dorobionym widokiem resource-lane jest rozważane, ale niepotwierdzone; alternatywą jest lekki komponent osi czasu karmiony propsami (zero domeny), łatwiejszy do widoku wiersz-per-podmiot i trybu edycji z przeciąganiem. Wybór konkretnego komponentu pozostaje otwarty. Sedno jest niezależne od tej decyzji: widok wiersz per podmiot, skala dzienna w MVP, model danych gotowy na skalę godzinową bez zmian.

10. Zależności i zgodność z regułami Open Mercato

- **Kierunek zależności:** `reservations` sięga po `planner` do odczytu (przez `tryResolve`, z degradacją gdy plannera brak). Zapis reguł zajętości przez komendy plannera (`planner.availability.create|update|delete` na `commandBus`), z uwzględnieniem optimistic locking.
- **Brak relacji ORM między modułami:** powiązania z `resources` / `planner` przez FK po id. `defineLink` tylko tam, gdzie query engine ma joinować przez granicę modułu w jednym zapytaniu (np. filtrowanie/sortowanie listy rezerwacji po kolumnach podmiotu).
- **Zapis przez komendy**, side-effecty po flush, CRUD przez `makeCrudRoute` z `indexer`, walidacja zod, filtrowanie zawsze po `organizationId` / `tenantId`.
Zgodnie z regułami architektonicznymi Open Mercato.

Warstwa zapisu/odczytu (komendy, CRUD, walidacja, indeksowanie) jest **oddzielona od czystego rdzenia** i zależy od konwencji konkretnego wdrożenia. Rdzeń (silnik + liczenie kalendarza) nie zależy od niej i nie wie o route ani o komendach. Dzięki temu rdzeń przenosi się bez zmian, a warstwa transportu jest dopasowywana lokalnie.